Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Agustín Yagüe y Jennifer Pérez
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

- **CONTENIDOS**
  - **Behaviour-Driven Development (BDD)**
  - **Testing - JUnit**
  - **Test-Driven Development**



¿Qué es Behaviour-Driven Development (BDD)?

BDD es una técnica que permite implementar una aplicación a través de la descripción de su comportamiento desde el punto de vista de los colectivos involucrados

Filosofía BDD

Business

Product Owner

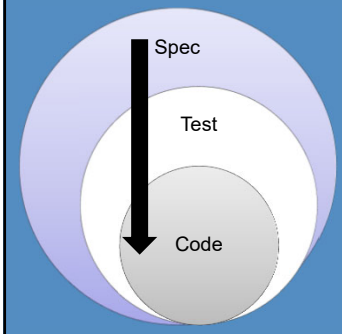
Production Support



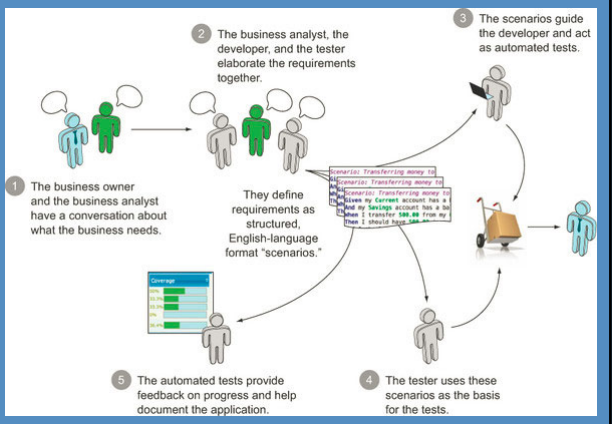
Developers

Quality Assurance

The BDD school of thought, Outside in



Outside in



Behaviour-Driven Development

- It is a testing technique joining both business and technique approaches
- Tests are represented describing application scenarios
- Describes the scenario in plain text using business domain
- Assists with levelling the story
- Tests are related to user stories and they are written following a syntax:

Given initial context
When event happens
Then outcomes achieved

Ventajas de BDD

- Alta colaboración porque fomenta la conversación
- *Feedback* temprano
- Documentación actualizada y viva
- Expresado en el lenguaje del cliente
- Criterios de aceptación ejecutables
- Aumenta el nivel de automatización
- Permite gestionar el nivel de entregas

BDD: User Stories y Escenarios

- **Criterios de Aceptación** definidos mediante los **escenarios** que pueden suceder durante la funcionalidad que define una user story

Title: Retirada de Efectivo
Descripción: **Como** cliente del banco **Quiero** retirar dinero en efectivo del cajero **Para** no tener que ir a la sucursal

Escenario 1: Cuenta con saldo

Escenario 2: Cuenta con saldo insuficiente

BDD: User Stories y Escenarios

- Especificación de escenarios:

Title: Retirada de Efectivo
Descripción: **Como** cliente del banco **Quiero** retirar dinero en efectivo del cajero **Para** no tener que ir a la sucursal

Escenario 1: Cuenta con saldo

Given una cuenta con saldo
given una tarjeta válida
given el cajero tiene dinero
When el cliente pide retirar dinero
Then se descuenta el dinero del saldo
then se entrega el dinero
then se expulsa la tarjeta

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

BDD: User Stories y Escenarios

- Especificación de escenarios:

Title: Retirada de Efectivo
Descripción: **Como** cliente del banco **Quiero** retirar dinero en efectivo del cajero **Para** no tener que ir a la sucursal

Escenario 2: Cuenta con saldo insuficiente

Given una cuenta con saldo insuficiente
given una tarjeta válida
given el cajero tiene dinero
When el cliente pide retirar dinero
Then se muestra mensaje de rechazo
then NO se entrega el dinero
then se expulsa la tarjeta

MDW: Metodologías Web

BDD: User Stories y Escenarios

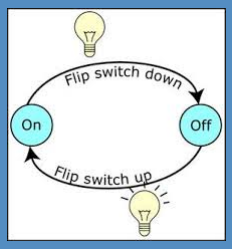
- Escenarios vistos como Transiciones de Estados:

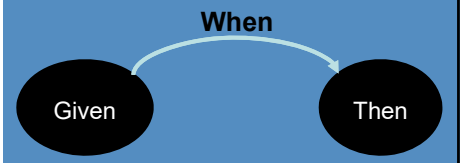
Escenario 1: Cuenta con saldo

Given una cuenta con saldo
given una tarjeta válida
given el cajero tiene dinero

When el cliente pide retirar dinero

Then se descuenta el dinero del saldo
then se entrega el dinero
then se expulsa la tarjeta

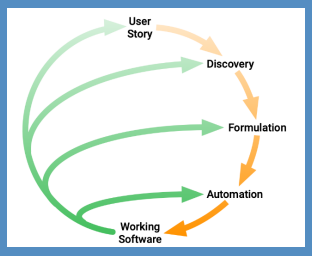




Las 3 prácticas de BDD

Las actividades de BDD se pueden resumir en 3 pasos de un proceso iterativo:

1. **Descubrir**: seleccionar una pequeña funcionalidad del Sistema (historia de usuario) y describirla en base a ejemplos/escenarios de cómo funciona. Descubre y acuerda los detalles de uso.
2. **Documentar** los escenarios mediante una formulación que pueda ser automatizada.
3. **Automatizar** la ejecución de las pruebas para comprobar el escenario y realizar la implementación del escenario.



De esta forma:

- Los escenarios sustituyen a la documentación
- El código está alineado con la documentación

BDD: Herramientas

- **Lenguaje**
  - Gherking
- **Framework**
  - cucumber

BDD: Gherkin

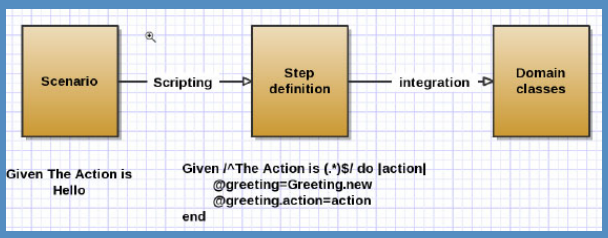
Gherkin DSL es un lenguaje con una gramática bien definida que permite especificar comportamientos en lenguaje natural

- Se basa en un conjunto reducido de palabras clave:
  - Feature
  - Rule
  - Example o Scenario
  - Given, When, Then, And, But
  - Background
  - Scenario Outline o Scenario Template
  - Examples

- Además hay un conjunto de tags complementarios
  - “”” para representar cadenas de caracteres
  - | para representar tablas de datos
  - @ anotaciones
  - # comentarios

BDD: Gherkin

- Las pruebas descritas con Gherkin constan de tres elementos:
  - **Features**: que definen el comportamiento del Sistema
  - **Scenarios**: que representan un comportamiento específico del Sistema.
  - **Pasos**: que representan las acciones que se describen el comportamiento



BDD: Gherkin

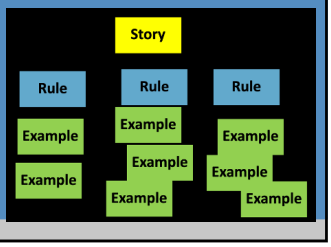
- **Feature**
  - Es una descripción de alto nivel de una funcionalidad del Sistema y que agrupa un conjunto de escenarios.
  - Una Feature tiene tres partes:
    - La palabra reservada **Feature**
    - Un nombre
    - Un descripción de la funcionalidad. Este texto es meramente informativo. La descipción acaba cuando se encuentra una de las palabras reservadas: Background, Rule, Example o Scenario Outline.
  - Examples:

    Feature: Credit card format validation.
    Credit card numbers must be exactly 16 characters.

BDD: Gherkin

- **Rule** (a partir de Gherkin 6)
  - Permite representar reglas de negocio que deben implementarse
  - Permiten complementar la información de una Feature.
  - Se usan para agrupar escenarios sobre las que se aplica la regla de negocio
  - Tiene que contener al menos un escenario.
  - Example:

    Feature: Credit card format validation.
    Rule: Credit card size
    Example: Longitud **is** 16
    Example: Longitud < 16



BDD: Gherkin

- **Example / Scenario**
  - A scenario is a concrete example that illustrates a business rule. It consists of a list of steps.
  - Scenarios can have as many steps as needed, but we recommend you keep the number at 3-5 per scenario.
  - If they become longer than that, they loose their expressive power as specification and documentation.
  - A scenario is also a test. As a whole, your scenarios are an executable specification of the system.
  - Scenarios follow the same pattern:
    - Describe an initial context (**Given**)
    - Describe an event (**When**)
    - Describe an expected outcome (**Then**)

    } **Steps**
  - Example:

    Scenario: Credit card number is too short

BDD: Gherkin

- **Scenario outline**
  - It is an scenario following the same pattern but with different values
  - Placeholders are represented by names between brackets <name>
  - Values are represented by tables where column names correspond to placeholder names
  - Each row represents an scenario
  - Example:

Scenario Outline: Celsius Conversion Success
As a user
I want to convert Celsius degrees to Fahrenheit
So that I can understand temperatures on each scale
Given I am in "TemperatureConverterActivity" screen
When I enter <***input_celsius***> Celsius into input with id "celsius"
Then I want to see <***output_fahrenheit***> Fahrenheit into field with id "fahrenheit"
Examples:

input_Celsius	output_fahrenheit
100	212.00
37.78	100.00

BDD: Gherkin

- **Steps**
  - A step typically starts with **Given, When, Then, And** or **But**.
  - If there are multiple Given or When steps underneath each other, you can use **And** or **But**.
  - Cucumber does not differentiate between the keywords, but choosing the right one is important for the readability of the scenario as a whole.
  - Steps are domain language phrases that we can combine to write scenarios.
  - They can either refer to:
    - **Context** of a scenario generally begin with the **Given** keyword, for example:
      Given the ninja has a third level black-belt
    - An **action** generally begin with the **When** keyword, for example:
      When attacked by a samurai
    - An **expected outcome** generally start with the **Then** keyword, for example:
      Then the ninja should engage the opponent

BDD: Gherkin

- **Given**
  - Given steps are used to describe the initial context of the system—the scene of the scenario. It is typically something that happened in the past.
  - When Cucumber executes a Given step it will configure the system to be in a well-defined state, such as creating and configuring objects or adding data to the test database.
  - The purpose of Given is to put the system in a **known state**.
  - Given steps should describe the state of the world before an interesting action. Avoid writing them as actions, but instead explain the state.
  - Do not try to include everything in a single Given statement
  - It's ok to have several Given steps (just use And or But for number 2 and upwards to make it more readable).
  - Use parameters to identify different paths
  - Example:

    Given a form with a field to introduce credit card numbers

BDD: Gherkin

- **When**
  - When steps are used to describe an event, or an action. This can be a person interacting with the system, or it can be an event triggered by another system.
  - It's strongly recommended you only have a **single** When step per scenario. If you feel compelled to add more it's usually a sign that you should split the scenario up in multiple scenarios.
    - If the When requires more than one steps mean that it is potentially complex
    - Makes more difficult to build the example table with multiple actions
  - Be sure that the When is not a precondition
    - When there is cash in the ATM dispenser AND I withdraw 100 € from my account
- Example:

  When I use the native keyboard to enter "123456"
  into text field number 1
  And I touch the "Validate" button

BDD: Gherkin

- **Then**
  - Then steps are used to describe an expected outcome, or result.
  - The step definition of a Then step should use an assertion to compare the actual outcome (what the system actually does) to the expected outcome (what the step says the system is supposed to do).
  - Then steps are post-conditions, expected observable outcomes of the primary action in the given context.
  - It is important that these are observable outcomes, something that the business users can actually understand and see. Otherwise, they won't be able to engage in specifying it.
  - Avoid internal state changes, though they may be easy to test, do not guarantee that the user of the system benefits in any way from the feature.
  - Example:

    Then I see the text "Credit card number is too short."

BDD: Gherkin

- **And, But**
  - Permiten reemplazar mútiples Given para hacer más legible la estructura de la especificación

    Example: Multiple Givens
    Given one thing
    Given another thing
    Given yet another thing
    When I open my eyes
    Then I should see something
    Then I shouldn't see something else
  - Podría quedar como:

    Example: Multiple Givens
    Given one thing
    And another thing
    And yet another thing
    When I open my eyes
    Then I should see something
    But I shouldn't see something else

BDD: Gherkin

- **Background**
  - Es útil cuando hay una serie de sentencias Given que se repiten en multiples escenarios de una Feature
  - Esos pasos no son esenciales para describir el scenario sino que son complementarios (equivalente al @Before de jUnit)
  - Permiten definir el contexto de los escenarios y pueden contener más de un Given
  - Se ubican siempre antes del primer Example / Escenario
  - Sólo puede haber un Background en cada Feature
  - Recomendaciones
    - No usar Background para definer estados complejos:
    - Debe de ser corta (más de 4 nos da pistas de que es una Feature compleja)
    - Personifica el Background

BDD: Gherkin

Background

Feature: Feedback when entering invalid credit card details

Background:
Given I have chosen some items to buy
And I am about to enter my credit card details

Scenario: Credit card number too short
When I enter a card number that's only 15 digits long
And all the other details are correct
And I submit the form
Then the form should be redisplayed
And I should see a message advising me of the correct number of digits

Scenario: Expiry date invalid
When I enter a card expiry date that's in the past
And all the other details are correct
And I submit the form
Then the form should be redisplayed
And I should see a message telling me the expiry date must be wrong

BDD: Gherkin

- **Paso de argumentos en Steps**
  - **Cadenas de caracteres ””**
  - **Números:** basta con escribir los dígitos
  - **Doc Strings**
    - Para pasar cadenas de caracteres largas “””
    - El texto tendrá que ir indentado (2 espacios) y se pasará como ultimo argumento de la definición del paso

```
Given a blog post named "Random" with Markdown body
  """
  Some Title, Eh?
  ===============
  Here is the first paragraph of my blog post. Lorem ipsum dolor sit amet,
  consectetur adipiscing elit.
  """
```


BDD: Gherkin

- **Paso de argumentos en Steps**

```
Feature: Búsqueda en Google
  Como usuario web, quiero buscar en Google para poder responder mis dudas.

  Scenario: Búsqueda simple en Google
    Given un navegador web en la página de Google
    When se introduce la palabra de búsqueda "pingüino"
    Then se muestra el resultado de "pingüino"
    And la página de resultados muestra el texto de Wikipedia
      """
      Nombre científico: Spheniscidae
      Clase: Aves
      """
```

BDD: Gherkin

- Argumentos en los pasos
  - Selección del lenguaje
    - # language: es

English Keyword	Spanish equivalent(s)
feature	Característica
background	Antecedentes
scenario	Escenario
scenarioOutline	Esquema del escenario
examples	Ejemplos
given	* Dado Dada Dados Dadas
when	* Cuando
then	* Entonces
and	* Y E
but	* Pero

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

BDD: Gherkin

```
Feature: Change PIN

  Customers being issued new cards are supplied with a Personal
  Identification Number (PIN) that is randomly generated by the
  system.

  In order to be able to change it to something they can easily
  remember, customers with new bank cards need to be able to
  change their PIN using the ATM.

  Scenario: Change PIN successfully
    Given I have been issued a new card
    And I insert the card, entering the correct PIN
    When I choose "Change PIN" from the menu
    And I change the PIN to 9876
    Then the system should remember my PIN is now 9876

  Scenario: Try to change PIN to the same as before
    Given I have been issued a new card
    And I insert the card, entering the correct PIN
    When I choose "Change PIN" from the menu
    And I try to change the PIN to the original PIN number
    Then I should see a warning message
    And the system should not have changed my PIN
```

MDW: Metodologías Web

BDD: Gherkin

```
Feature: Change PIN

  As soon as the bank issues new cards to customers, they are
  supplied with a Personal Identification Number (PIN) that
  is randomly generated by the system.

  In order to be able to change it to something they can easily
  remember, customers with new bank cards need to be able to
  change their PIN using the ATM.

  Background:
    Given I have been issued a new card
    And I insert the card, entering the correct PIN
    And I choose "Change PIN" from the menu

  Scenario: Change PIN successfully
    When I change the PIN to 9876
    Then the system should remember my PIN is now 9876

  Scenario: Try to change PIN to the same as before
    When I try to change the PIN to the original PIN number
    Then I should see a warning message
    And the system should not have changed my PIN
```


CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

BDD: Gherkin

Data Tables

```
Given a User "Michael Jackson" born on August 29, 1958
And a User "Elvis" born on January 8, 1935
And a User "John Lennon" born on October 9, 1940
...
```

```
Given these Users:
  | name            | date of birth   |
  | Michael Jackson | August 29, 1958 |
  | Elvis           | January 8, 1935 |
  | John Lennon     | October 9, 1940 |
```

MDW: Metodologías Web

BDD: Gherkin

```
Feature: Withdraw Fixed Amount

  The "Withdraw Cash" menu contains several fixed amounts to
  speed up transactions for users.

  Scenario: Withdraw fixed amount of $50
    Given I have $500 in my account
    When I choose to withdraw the fixed amount of $50
    Then I should receive $50 cash
    And the balance of my account should be $450

  Scenario: Withdraw fixed amount of $100
    Given I have $500 in my account
    When I choose to withdraw the fixed amount of $100
    Then I should receive $100 cash
    And the balance of my account should be $400

  Scenario: Withdraw fixed amount of $200
    Given I have $500 in my account
    When I choose to withdraw the fixed amount of $200
    Then I should receive $200 cash
    And the balance of my account should be $300
```


CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

BDD: Gherkin

```
Feature: Withdraw Fixed Amount

  The "Withdraw Cash" menu contains several fixed amounts to
  speed up transactions for users.

  Scenario Outline: Withdraw fixed amount
    Given I have <Balance> in my account
    When I choose to withdraw the fixed amount of <Withdrawal>
    Then I should receive <Received> cash
    And the balance of my account should be <Remaining>

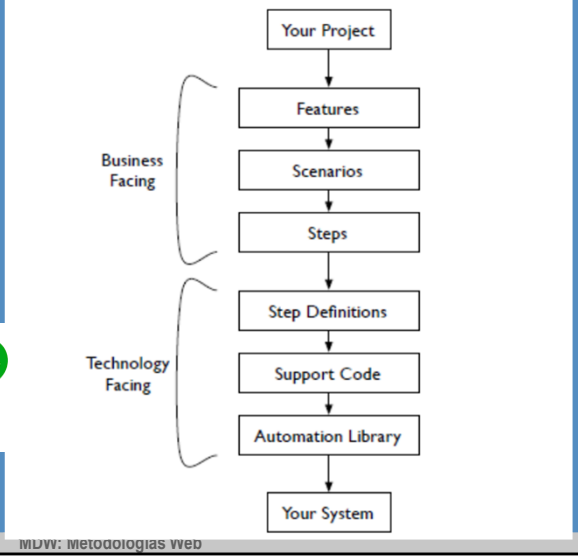
    Examples:
      | Balance | Withdrawal | Received | Remaining |

      | $500    | $50        | $50      | $450      |
      | $500    | $100       | $100     | $400      |
      | $500    | $200       | $200     | $300      |
```

BDD: Proceso

Gherking

cucumber



BDD: Cucumber

Maven™

m pom.xml

```
<dependencies>
   <!-- Source: https://mvnrepository.com/artifact/io.cucumber/cucumber-java --
>
   <dependency>
      <groupId>io.cucumber</groupId>
      <artifactId>cucumber-java</artifactId>
      <version>7.34.2</version>
      <scope>compile</scope>
   </dependency>
</dependencies>
```

https://cucumber.io/docs/guides/10-minute-tutorial/

BDD: Cucumber

Ejecución

```
Feature: Is it Friday yet?
  Everybody wants to know when it's Friday

  Scenario: Sunday isn't Friday
    Given today is Sunday
    When I ask whether it's Friday yet
    Then I should be told "Nope"
```

https://cucumber
tutorial/

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running hellocucumber.RunCucumberTest
Feature: Is it Friday yet?
  Everybody wants to know when it's Friday

  Scenario: Sunday isn't Friday        # hellocucumber/is_it_friday_yet.feature:4
    Given today is Sunday              # null
    When I ask whether it's Friday yet # null
    Then I should be told "Nope"       # null

Undefined scenarios:
hellocucumber/is_it_friday_yet.feature:4 # Sunday isn't Friday

1 Scenarios (1 undefined)
3 Steps (3 undefined)
0m0.040s
```


BDD: Cucumber

Steps Definition

```
Feature: Is it Friday yet?
  Everybody wants to know when it's Friday

  Scenario: Sunday isn't Friday
    Given today is Sunday
    When I ask whether it's Friday yet
    Then I should be told "Nope"
```

https://cucumber.
tutorial/

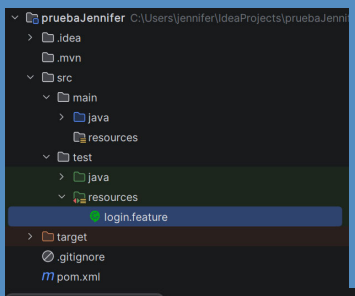
```
You can implement missing steps with the snippets below:

@Given("today is Sunday")
public void today_is_Sunday() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

@When("I ask whether it's Friday yet")
public void i_ask_whether_it_s_Friday_yet() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

@Then("I should be told {string}")
public void i_should_be_told(String string) {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}
```

BDD: Cucumber



login.feature

```
Feature: Login User
  As a user I want to login in the web page to access to the functionality
  Scenario: Successful login
    Given open the login window
    When the user enters correctly the username and password
    And clicks the ok button
    Then the login windows is closed
    And the homepage is open
```


BDD: Cucumber

```
Undefined scenarios:
  1) Successful login # file:///C:/Users/jennifer/IdeaProjects/ejemplo2/src/test/resources/Login.feature:

1 scenarios (1 undefined)
5 steps (4 skipped, 1 undefined)
0m 0.155s

You can implement missing steps with the snippets below:

@Given("open the login window")
public void open_the_login_window() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

@When("the user enters correctly the username and password")
public void the_user_enters_correctly_the_username_and_password() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}
```

BDD: Cucumber

```
@When("clicks the ok button")
public void clicks_the_ok_button() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

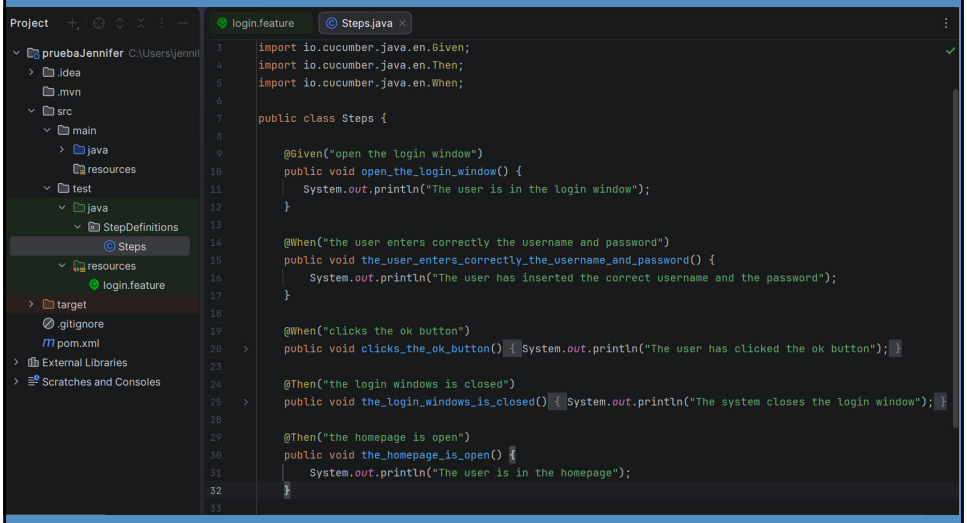
@Then("the login windows is closed")
public void the_login_windows_is_closed() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

@Then("the homepage is open")
public void the_homepage_is_open() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}


Process finished with exit code 1
```


CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

BDD: Cucumber



MDW: Metodologías Web

BDD: Cucumber

Support Code

```java
package StepDefinitions;

import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;

public class Steps {

    @Given("open the login window")
    public void open_the_login_window() {
       System.out.println("The user is in the login window");
    }

    @When("the user enters correctly the username and password")
    public void the_user_enters_correctly_the_username_and_password() {
        System.out.println("The user has inserted the correct username and the password");
    }

    @When("clicks the ok button")
    public void clicks_the_ok_button() { System.out.println("The user has clicked the ok button"); }

    @Then("the login windows is closed")
    public void the_login_windows_is_closed() { System.out.println("The system closes the login window"); }
```


BDD: Cucumber

login.feature | Steps.java | pom.xml (pruebaJennifer)

```
Feature: Login User
  As a user I want to login in the web page to access to the functionality
  Scenario: Successful login
    Given open the login window
    When the user enters the username and password
    And clicks the ok button
    Then the login windows is closed
    And the homepage is open
```

Done: Scenarios 1 of 1 (807 ms)

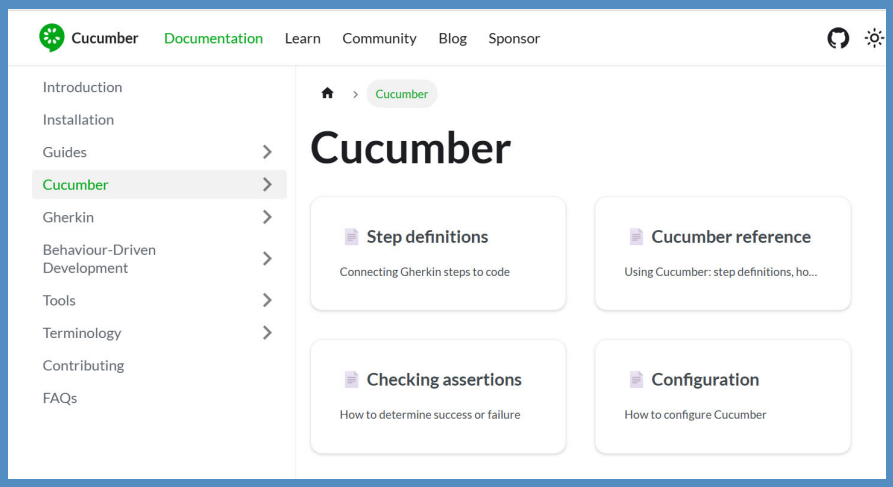
```
"C:\Program Files\Java\jdk-25\bin\java.exe" ...
Testing started at 1:11 ...
The user is in the login window
The user has inserted the username and the password
The user has clicked the ok button
The system has closed the login window
The user is in the homepage

1 scenarios (1 passed)
5 steps (5 passed)
0m 0.131s

Process finished with exit code 0
```

Support Code: Test Definition Execution

Bibliografía



Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Agustín Yagüe y Jennifer Pérez
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos